

MVC architecture

Model View Controller (MVC) is a software design pattern used for the development of Web applications. It is made up of the following three parts:

- Model – the lowest level, which is responsible for maintaining data.
- View – responsible for displaying all or a portion of the data to the user.
- Controller – controls the interactions between the Model and View.

Model

The model contains the data to be displayed, as well as data to be collected in any input fields or forms. Additionally, it may contain functions that are called based on user input or other activities. A model in Angular is just a plain JavaScript object. It can be a primitive type such as a string, number, Boolean or a complex type such as an object.

View

View is nothing but how you present the data in a particular format. It may be triggered by the controller's decision. The view in an AngularJS application is created using HTML. View is what users see. It binds data from the model inside your controller, and will get updated automatically if there are model changes from the controller, since Angular has two-way data binding. There is no need to write additional code to achieve this.

Controller

Controller is the place where the application logic resides. It is simply formed by JavaScript classes. In it, we can also call other components to work with. The controller is where the model lives.

The controller for a view is used to pull together the model used by the view and handle the input from the consumer of the view. In AngularJS, the controller is resolved by name, from the ng-controller directive.

```
<html ng-app="testApp">
<head>
<title>MVC Architecture code</title>
<script SRC="https://ajax.googleapis.com/ajax/libs/angularjs/1.2.13/angular.js">
</script>
<script type="text/javascript">
//Creating controller here
var app = angular.module('testApp', []);
app.controller('callcontroller', function($scope) {
    //Creating model here
    $scope.employee = {
        'Name' : 'ABC',
        'Address' : '12-13-283/A1',
```

```
'Email' : 'abc@xyz.com'
    }
});
</script>
</head>
<body>
<p>Displaying model data in view through controller</p>
<div ng-controller="callcontroller">
<h3>{{ employee .Name }} </h3>
<h3>{{ employee .Address }} </h3>
<h3>{{ employee .Email }} </h3>
</div>
</body>
</html>
```

The output displays the model data in view, through the controller:

```
ABC
12-13-283/A1
abc@xyz.com
```

Filters

Filters are used to modify the data and can be used in expressions or directives, using the pipe character. The following is a list of commonly used filters.

Name	Description
uppercase	Converts text to upper case.
lowercase	Converts text to lower case.
currency	Converts text to a currency format.
filter	Filters the array to a subset of it, based on the provided criteria.
orderBy	Orders the array based on the provided criteria.

Building your own directives

AngularJS allows you to build your own custom directives, which are nothing but an extension to the HTML. They are defined using the 'directive' function, and simply replace the element for which they are activated. AngularJS finds the matching elements and processes them once, using its *compile()* method; it then processes the elements using the *link()* method of the custom directive based on the scope. It supports the creation of custom directives for the following types of elements.

- Element directives: It activates once a matching element is encountered.
- Attribute: It activates once a matching attribute is encountered.
- CSS: It activates once a matching CSS style is encountered.
- Comment: It activates once a matching comment is encountered.